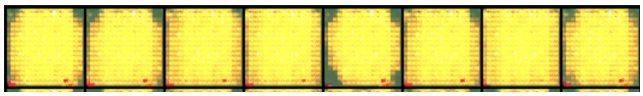
Optimizing DCGAN for Image Generation on Small Datasets

Simon Yan, Kenneth Tan, Suhrit Padakanti
UCSB CS190I: Generative AI Spring 2025

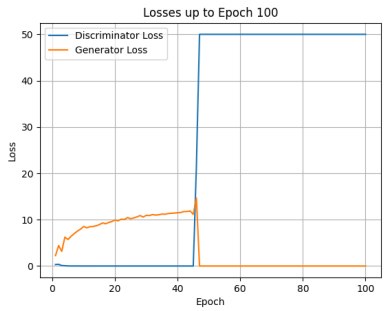
Abstract
DCGAN is an improvement of GANs for image generation through the addition of convolutional layers. However, with small datasets it will be difficult to generalize and produce unique outputs. In this project, we look at combining multiple different strategies to optimize DCGAN for more unique and realistic emoji generated outputs on a small dataset.

Dataset
The goal of this project was to generate good images with a small dataset. GANs typically work well with large datasets but we wanted to test different ways to improve training with a small dataset. We came across emojis as our main dataset because of the limited number of them as well as the similarities between each emoji. Each emoji we used in our dataset had the typical round head with some sort of face on it(eyes and mouth). Our dataset of emojis came in at around ~98 images for 6 different systems: Apple, Facebook, Google, Samsung, Twitter, and Windows. This gives us a little under 600 images to work with.

Problems with Base DCGAN
When training the base DCGAN on our initial dataset, we ran into many problems since DCGAN, and most other GANs, are not meant to train on a smaller dataset. We realized that the base DCGAN was mode collapse in the generator and an overpowering discriminator.
When first running the base DCGAN, the images were blobs of yellow, not changing in any way as the epochs progressed.



As we can see the images produced are junk and if you look closely, most blobs are similar in shape. This led to the following changes.

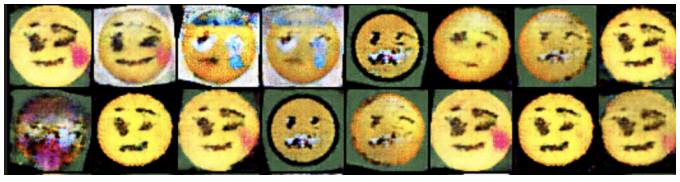


You can see that the discriminator loss is at 0 very early on and then jumps to 50 for some reason. Also the generator loss rises and then hits 0 abruptly after when the discriminator loss goes to 50.

Hyperparameter Changes
The first hyperparameter change was lowering the learning rate of the overall model, discriminator and generator. This was done in hopes that the discriminator would not learn super quick and hit a loss of 0. Another hyperparameter we played around with was batch size. We found that as we tested 128, 64, 32, and 16 batch sizes, the smaller the batch size, the better outputs we got. In turn, it also took longer to train. We believe that the better outputs and longer training times correlate to the fact that smaller batch sizes induce more noise and more frequent updates of the weights. The last hyperparameter that was experimented on was the number of epochs. Our training ranged from 100-2500 epochs. We found that 250- 300 epochs was sufficient enough to produce good images for this project.

Training Modifications
To help the base DCGAN, we decided to add a few changes. To list them, we first added label smoothing. This made it so that real images would be represented as 0.9 rather than 1.0, making the discriminator never 100% sure that the image was real. This helped lower the discriminator's confidence and greatly reduced the chance of vanishing gradients.
The second modification we added was lowering the discriminator's learning rate. This made the discriminator weaker, in turn making the generator stronger to avoid mode collapse.
Finally, we also added some noise for the real images when inputted into the discriminator. The randomness of the noise discourages overfitting from the discriminator and improves generalization and training stability.

Data Augmentation
In addition to modifying the training, we also added image augmentation into our dataset. This is to increase the training data for the GAN since we started out with a small dataset. The idea of data augmentation for our situation was to change the emojis slightly in ways that wouldn't affect how they are perceived. Our augmentations included small rotations, changing "color"(brightness, contrast, saturation, hue), and slight translations and scaling. For each image, we outputted 10 augmented versions. From this, we saw great improvement in the model's outputs, but it wasn't perfect.



We can see that some generated images are still very similar, but now they actually look like emojis rather than blobs. We

needed more optimizations and this time, we looked to modify DCGAN itself.

Spectral Normalization

The first optimization strategy we implemented to avoid mode collapse is spectral normalization. Spectral normalization prevents the discriminator of the DCGAN from exploding by bounding and stabilizing its training. The spectral norm is calculated by finding the single value decomposition (SVD) of a weight matrix, which is the maximum stretch on a vector when multiplied.

$$W_{SN} = \frac{W}{\sigma(W)}$$

To spectrally normalize the weight matrix, every value in the matrix is divided by its spectral norm. This normalizes the weight matrix, so now the maximum stretch on a vector that is multiplied against the normalized matrix is 1. This is called 1-Lipschitz continuous, and by having the matrix only be able to stretch a vector by up to one times its length prevents the discriminator from having huge gradients or overfit.

$$|f(x) - f(y)| \leq |x - y| \; or \; ||W_{norm}(x)|| \leq ||x||$$

Results of DCGAN Outputs with Spectral Normalization

The results prove to be much better than the base DCGAN model.

Historical Averaging

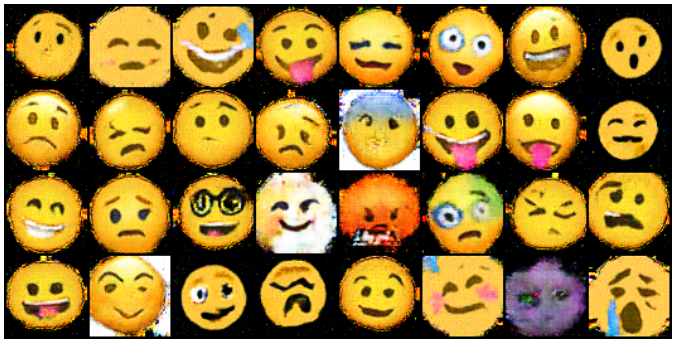
Although the discriminator is training more stable and providing actually good feedback to the generator, the generator of the DCGAN is still unstable and adapts too much to short-term discriminator feedback. To prevent this, we add regularization to optimize the loss function of the generator such that it would not make such big updates between epochs. Historical averaging is a regularization technique that uses exponential moving average (EMA) so updates in the current epoch take into account values from previous epochs. EMA gives more weight to recent values and less to values more in the past so that updates will not be stuck on changes from the early in training and still make necessary updates that align closer to what was recently changed in the generator to better match feedback from the discriminator.

$$Loss\ func + \lambda \cdot ||\Theta_t - \Theta_{t-1}||^2$$

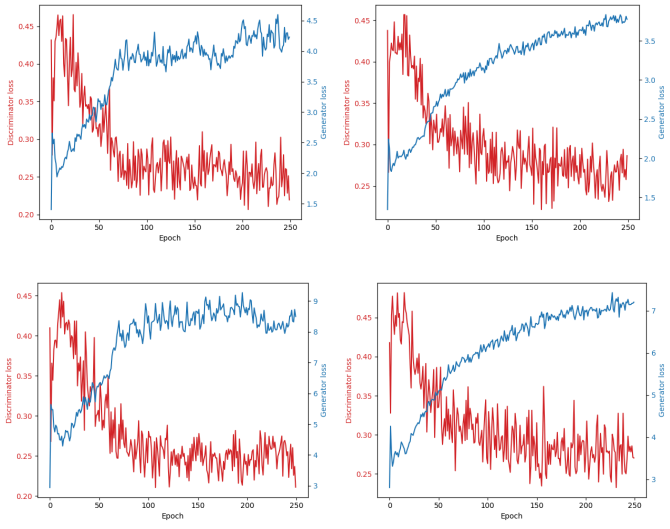
Through implementing this technique, the generator will not have steep updates based on noisy short-term feedback, but smoother updates built off of recent epochs.

Results of DCGAN Outputs with Historical Averaging

Results of DCGAN Outputs with SN and HA



Loss Graphs



Top left: Base DCGAN Loss
Top right: DCGAN with Spectral Normalization Loss
Bottom left: DCGAN with Historical Averaging Loss

Bottom right: DCGAN with SN and HA Loss

GAN loss graphs are difficult to interpret the prediction accuracy of the model as they measure the adversarial push and pull between the generator and discriminator. While we do look at the loss graphs, we will also look at the quality of images generated to gauge how well our modifications optimize DCGAN for generating images on small datasets.

Between the base DCGAN model loss and the DCGAN with spectral normalization model loss, it can be observed that by constraining the discriminator to a Lipschitz constant, it gives more consistent gradient feedback to the generator, allowing it to make smaller and more stable updates. The overall trend of the DCGAN with spectral normalization is also still positive, a sign that the generator is yet to collapse in 250 epochs. The generator does see small collapses in the base model, at around 100 and 150 epochs.

The bottom right graph, which is the loss for DCGAN with historical averaging, has double the generator loss as the base DCGAN model because of the regularization penalty that is added to the loss calculation whenever the loss strays too far from previous epoch weights. The overall shape and trend of the generator is still similar to the base DCGAN model as it Still learns from the same adversarial feedback from the generator. However, it can be observed that the updates in the generator are slightly more consistent compared to the base DCGAN model.

The DCGAN with both spectral normalization and historical averaging is as expected. It combines the observations that can be made from the DCGAN with just spectral normalization and the DCGAN with just historical averaging. The spectral normalization to the discriminator bounds it to provide consistent feedback, as seen in the smaller updates in the bottom right graph that are similar to the top right graph, and the historical averaging is an added regularization penalty to the generator loss that increases the loss through the function in the **Historical Averaging** section in the previous page.

Improving Image Quality

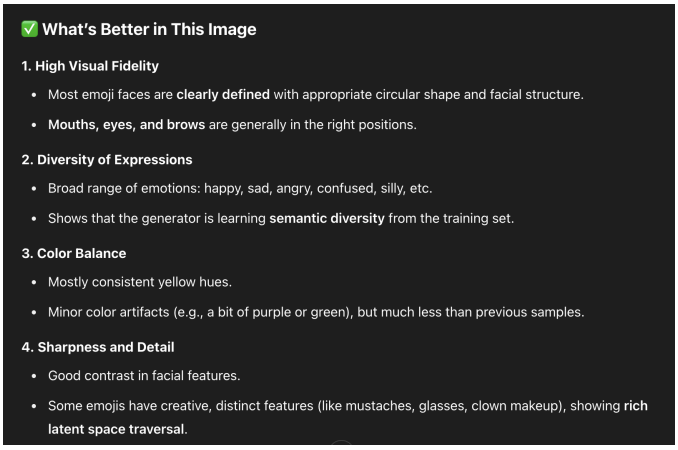
Implementing spectral normalization and historical averaging has stabilized the DCGAN training, but that does not optimize the quality of generated images. Standard convolutional layers are great at identifying local patterns like the texture of a smile but struggle to model long-range dependencies like the left eye in relation to the right eye of an emoji.

Our solution was to add a self-attention mechanism into our DCGAN architecture. For each pixel generated, the layer calculates an “attention map” that weighs how much focus to put on every other pixel in the image. This allows the generator to coordinate features across the entire image to ensure they are structurally consistent. This can be thought of as an artist stepping back from their canvas to see the whole composition before going back to paint a new feature in their artwork.

Analyzing Image Quality

Since GAN loss graphs are inherently difficult to use to analyze prediction accuracy, we will look into other methods for comparing the generated outputs to the original dataset images. There are methods like FID score and CLIP similarity score to analyze the likeness between original images and generated images using statistical analysis.

In our project, we used OpenAI ChatGPT to analyze different images and give us the best one based on accuracy and quality of images. ChatGPT was able to compare between our image outputs for the different combinations of modifications and give nuanced feedback. Based on ChatGPT’s response, the images generated with a DCGAN model with spectral normalization and historical averaging produced the best images, although still not as good as the original dataset. ChatGPT was able to identify the differences in diversity between our outputs and color ranges, which are not able to be shown in the loss graphs we plotted.



✅ **What’s Better in This Image**

1. High Visual Fidelity

- Most emoji faces are **clearly defined** with appropriate circular shape and facial structure.
- **Mouths, eyes, and brows** are generally in the right positions.

2. Diversity of Expressions

- Broad range of emotions: happy, sad, angry, confused, silly, etc.
- Shows that the generator is learning **semantic diversity** from the training set.

3. Color Balance

- Mostly consistent yellow hues.
- Minor color artifacts (e.g., a bit of purple or green), but much less than previous samples.

4. Sharpness and Detail

- Good contrast in facial features.
- Some emojis have creative, distinct features (like mustaches, glasses, clown makeup), showing **rich latent space traversal**.

References and Resources

DCGAN from scratch - https://www.youtube.com/watch?app=desktop&v=IZtv9s_Wx9I&t=578s
https://github.com/simony05/cs190i_finalproj
https://github.com/k-nn-tht/emoji_gan
https://github.com/suhritpadakanti/CS-190I
Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. "Spectral Normalization for Generative Adversarial Networks." arXiv:1802.05957, 2018.
Radford, A., Metz, L., & Chintala, S. (2015). *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. arXiv:1511.06434